I-TREA II: a GPU evolutionary algorithm coupled with MIP solvers — interim report on development (phase 1)

Berk Orbay
berk.orbay@tideseed.com*

17 October 2026

Abstract

On an NVIDIA DGX Spark, I-TREA II couples a GPU evolutionary algorithm (EA) with SCIP, exchanging solutions both ways; the EA also takes NVIDIA cuOpt's solutions. Its population keeps a capped share of infeasible solutions whose objective beats the incumbent's. This report covers development on 17 public instances that were chosen and tuned on (120-s runs). On the 11 instances hard for cuOpt alone, I-TREA II's primal integral (0: optimal at once; 1: no useful solution) was lower than with cuOpt alone (same cores and GPU) on 7 and higher on 3 (mean 0.690 to 0.550, four seeds). On the 15 hard for SCIP alone (one core, 6–7 October), it was lower on 12 and tied on 3 (0.889 to 0.448; cuOpt alone on these 15: 0.517). In the largest loss, the EA delays cuOpt's first solution from 6.5–12.2 to 65 s. Unseen instances are tested next.

What this means for SCIP and cuOpt users. In short: on instances hard for a solver alone, there is a real chance of a large gain, but not on every instance (Table 1). Most of the gain over SCIP alone is what cuOpt alone also reaches on the same five cores and GPU (mean P on SCIP's hard set: SCIP alone 0.889, cuOpt alone 0.517, I-TREA II 0.448). cuOpt on the whole machine, which is stronger (Section 6), has not yet been compared. Over all 17 instances, I-TREA II was better than cuOpt alone on 8 and worse on 8. On the 6 outside cuOpt's hard set it was worse on 5, by at most 0.022 in P except on `chromaticindex1024-7` (0.08 to 0.54), where the EA delays cuOpt's first solution (Section 4.3). On `doon` and `rail01`, cuOpt alone found a solution on 2 and 1 of 4 seeds, I-TREA II on none. All runs lasted 2 minutes, on instances we tuned on.

1 Introduction

Users of MIP solvers in planning and scheduling often need a good feasible solution within a fixed time, not a proof of optimality. The primal integral $P(T)$ measures how soon a solver gets there: it averages the gap to the best known value over the time budget T [4]. Lower is better; $P = 1$ if no solution was found.

GPUs now sit beside CPU solvers on small machines, and they suit population search: many candidate solutions can be repaired at once. NVIDIA cuOpt already runs GPU heuristics and a

*Designed and carried out with AgentRA (Agent Research Assistant; mainly Claude Opus 5.5, working through Claude Code), under the author's direction; see Appendix E. Every result of this study was produced by the code, configurations and job definitions in the project repository (to be made public with the final version).

Table 1: Change in the primal integral $P(120)$ (lower is better; 0 = optimal at once, 1 = no solution or a gap of 1 throughout) when a solver runs inside I-TREA II, on that solver's hard set: the instances where it alone had $P(120) \geq 0.5$ (Table 2). Seed means: four seeds for I-TREA II and cuOpt alone, run back to back; two for SCIP alone. 2-minute runs on one DGX Spark; p: Wilcoxon signed-rank. In the SCIP row, the bracket gives the change from SCIP alone to cuOpt alone, and the largest gains are among the instances where I-TREA II also beats cuOpt alone. ca: the unit-commitment case; full names in Appendix B.

solver alone	hard set	change in mean P	largest gains	losses
SCIP (one core; 6–7 October)	15 of 17	−50 % (cuOpt alone: −42 %); 12 better, 0 worse; $p = 0.0005$	neos-960392 1.00 → 0.04; ca 1.00 → 0.26	none
cuOpt (same 5 of 20 cores and GPU)	11 of 17	−20 %; 7 better, 3 worse; $p = 0.03$	neos-960392 0.58 → 0.04; ca 0.56 → 0.26; item_placement 0.49 → 0.22	doon 0.86 → 1.00; toguru 0.55 → 0.61; rail01 0.98 → 1.00

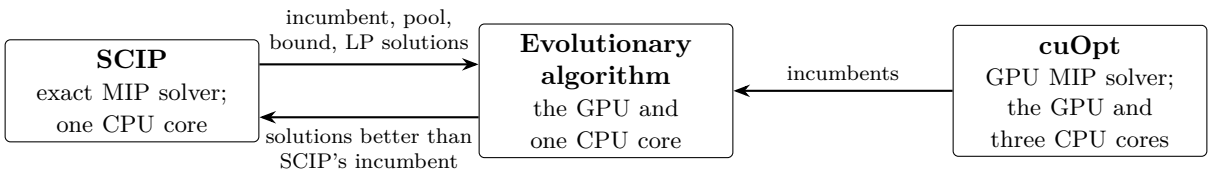


Figure 1: The coupled system. SCIP and the evolutionary algorithm exchange solutions in both directions; cuOpt only sends. The algorithm forwards to SCIP every solution better than SCIP's incumbent, cuOpt's included.

GPU population beside its own branch-and-bound [10, 32]. On the machine used here, running cuOpt beside SCIP or HiGHS without exchange lowered the primal integral at 60 s by 21 % and 30 % [35]; there, a CPU population exchanging solutions with SCIP gave no detectable gain.

I-TREA II moves the population to the GPU, as an evolutionary algorithm (EA), and couples it with exact solvers while they run; during development it grew to three processes on one NVIDIA DGX Spark (Figure 1). From the first I-TREA [34] it keeps one principle: infeasible solutions whose objective is better than the best feasible one keep a capped share of the population.

The aim is not to be better on average, but to help each solver where it is weakest, with the average as a guard. This report covers phase 1: development on 17 public instances at 120 s, up to a checkpoint that decides whether to continue. Its question: does the coupled system lower the primal integral on instances hard for cuOpt alone or SCIP alone, without being worse than cuOpt alone on average?

The report gives the method fixed after phase 1 (Section 2), what each tested change did and the checkpoint result with its main loss (Section 4), and a benchmark harness that gives every compared GPU solver the same cores and GPU (Appendix C).

2 Method

This section describes the configuration fixed after phase 1 (file and parameters: Appendix D): three processes on one machine, two of which share the GPU (Figure 1).

Processes and exchange. SCIP 10 [19] runs with one thread on one CPU core. The EA runs on a second core and the GPU, and cuOpt 26.08 [31] with three threads on three smaller cores and the same GPU. All three start together once each has read the instance. SCIP sends the EA its incumbent, up to eight pool solutions, its proven bound, and LP solutions of its tree nodes, including infeasible ones better than the incumbent. The EA sends SCIP, through a heuristic plug-in, every solution better than SCIP's incumbent. cuOpt streams its incumbents to the EA but receives nothing: during the solve, cuOpt 26.08 accepts outside solutions only through a callback that switches off its presolve, which in our predecessor study made a two-way exchange with cuOpt lose 14.7 % [35].

Population and ranks. The EA keeps up to 256 individuals in three ranks. *Rank 1* protects the best 10 % of the feasible solutions. *Rank 0* holds up to 10 % infeasible solutions whose objective is better than the incumbent, the best of the EA's, SCIP's and cuOpt's. A Rank 0 member whose objective is better than SCIP's proven bound cannot become feasible at that value, so it is removed. *Rank 2* fills the rest from the new candidates, ranked by objective and constraint violation, and tournaments prefer Rank 1, then Rank 0, then Rank 2.

Variation and repair. Children come from crossover, which mostly keeps the values both parents agree on, and from mutation of about four integer variables. A feasibility-jump local search [27] then repairs each child on the GPU, one thread block per child. It moves one variable at a time to reduce a weighted row violation; once an incumbent exists, an objective row keeps the repair from giving up objective value. A *repair clock* picks each generation's repair depth from the objective gain per second measured at each depth.

LP solutions and the last two changes of phase 1. A HiGHS process [18, 20] on the EA's core solves the LP relaxation once, and repaired roundings of its solution join the population. Two changes make better use of that single core:

- **LP decoding.** A second HiGHS process fixes the integer values of up to four candidates per generation and re-optimises their continuous variables, once the LP relaxation has finished.
- **Yielding wait.** By default the EA's host thread spins while it waits for the GPU. With CUDA's yield flag it gives its core to any other runnable process; in a check, a competing process got the whole core instead of about half.

3 Experimental setup

Phase 1 is development: the method was changed and judged openly on a small set of instances, so every number in Section 4 is descriptive. Table 2 defines the terms used from here on.

Machine. One NVIDIA DGX Spark: a GB10 with 10 Cortex-X925 and 10 Cortex-A725 CPU cores, a Blackwell GPU and 121 GiB of memory shared by CPU and GPU. The software is SCIP 10 through PySCIPOpt 6.2.1, HiGHS 1.15.1, cuOpt 26.08 and CuPy 14.2.0 on CUDA 13. No other GPU process ran beside the coupled system. Runs start only below temperature gates, and a run whose GPU clock drops is voided and repeated (Appendix C).

Table 2: Terms used in this report.

term	meaning
primal integral P	$P(120)$: the gap $\lvert v-r\rvert/\max(\lvert v\rvert,\lvert r\rvert)$ of the best checked solution value v to the reference r (1 before the first solution or if the signs differ), averaged over the 120-s budget.
coupled system	the method of Section 2; its P is that of the best solution held by any of its processes.
cuOpt alone	cuOpt 26.08 on the coupled system's share of the machine (5 of its 20 CPU cores and the GPU), run in the same window.
SCIP alone	SCIP 10 on one CPU core, two seeds, run on 6–7 October.
bounded difference d	per instance, on seed means: $(P_{\text{alone}}-P_{\text{coupled}})/\max(P_{\text{alone}},P_{\text{coupled}},0.001)$, in $[-1,1]$; positive when the coupled system has the lower P.
hard set	the instances where a solver alone has $P \geq 0.5$: 11 for cuOpt, fixed on earlier runs of cuOpt alone with three cores; 15 for SCIP, on the compared runs (Section 6).
hint	mean P of the coupled system over that of cuOpt alone, on cuOpt's hard set; present if at most 0.95. It only decides whether testing continues.
guard	mean d against cuOpt alone over all 17 instances; holds if at least -0.05.
harm check	per instance, the final configuration against the one before LP decoding and the yielding wait (13 October); flags if their d_i (as d) is below minus the instance's seed-to-seed band (at least 0.05).

Instances. We used 17 public instances (Appendix B). Nine were chosen after a screening run of SCIP alone. Seven are MIPLIB 2017 instances [16] on which SCIP held a large gap or found no solution; one is a PGLib-UC unit-commitment case [21, 24] and one an ML4CO item-placement instance [15]. Eight MIPLIB instances from other families were added later. All 17 were chosen by looking at results, the method was tuned on them, and they are excluded from later test sets. References are MIPLIB's optimal values and provisional best known values for the other two.

Arms. Several processes can do better than one simply by being several. cuOpt alone therefore gets the coupled system's whole machine share: two Cortex-X925 cores, three Cortex-A725 cores and the GPU. Both arms of each instance and seed ran back to back in one window, under the same memory caps. SCIP alone had fewer resources: one core, no GPU, run on 6–7 October. Only solutions that pass an independent exact checker count in P.

Statistics. We report mean d with a 95 % percentile bootstrap interval over instances (10,000 resamples) [12] and the Wilcoxon signed-rank test on d (ties dropped). The checkpoint result uses seeds 0–3 (SCIP alone: 0–1); development tests used one to six seeds.

How changes were judged. Each change was tested against the unchanged configuration in the same window, under a dated record with a written reading rule. It was kept only if its checks all passed: validity, the intended mechanism, no harm per instance, and the hint. Most rules were fixed before any run was read; Appendix A lists the exceptions. A crashed run counts for its arm, scored up to the crash.

The checkpoint. At the end of phase 1 the author decides whether to go on to unseen instances, reading the hint, guard and harm check under rules fixed before any checkpoint run was read.

4 Results

All numbers are descriptive, interim development data (17 instances, 120-s runs, 6–16 October).

Table 3: What phase 1 changed. Tests are numbered as in Appendix A; diagnostics and checkpoint readings (tests 3, 13, 14, 16–18) appear only there. Two-process: SCIP and the EA without cuOpt. d: mean bounded difference [95 % interval], positive in favour of the change.

test	change	effect	kept?
1	SCIP streams LP solutions of its tree nodes to the EA (two-process)	d +0.05; 3 better, 1 worse	yes[a]
2	repair clock: repair depth set per generation (two-process)	`item_placement` better on 4 of 4 seeds, d +0.50	yes[b]
4	cuOpt feeds its solutions to the EA (pilot: 7 instances, one seed)	against SCIP alone 7 better, 0 worse; against cuOpt alone 3 better, 4 worse	yes
5	Rank 1 chosen by cuOpt's diversity rule (two-process)	d −0.033 $[-0.122, +0.050]$	no
6, 12	LP decoding	two-process d +0.219 $[+0.041, +0.429]$; three-process: P ratio 0.758 on 2 targets	yes
7	the EA backs off the GPU while it adds nothing	`chromaticindex1024-7` P 0.545 → 0.091, but three checks that it worked as intended failed	no
8	the final harness's two cuOpt settings (Appendix C)	no harm on the 3 instances where they can act	yes
9	the EA backs off only until cuOpt's first solution	`chromaticindex1024-7` d +0.030 (needed $\geq +0.30$)	no
10	blocking wait on the GPU	the EA's generations fell to 0.82 of the control	no
11, 15	yielding wait on the GPU	first test fell short of the minimum number of EA generations; a follow-up passed	yes[c]

[a]Kept on without a rule fixed before its runs; not credited with a gain. [b]With two stated deviations from its own rules. [c]The follow-up loosened one bar after the failure was known.

4.1 What phase 1 changed, and what helped

Phase 1 kept six of the ten changes it tested (Table 3). The largest change was structural. On eight new instances, the two-process coupling of SCIP and the EA had a higher P than cuOpt alone on five and a lower one on one (mean d -0.51 $[-0.84, -0.12]$; test 3). So cuOpt joined as a third process instead of re-implementing cuOpt's heuristics inside the EA (test 4).

The last two changes kept, LP decoding and the yielding wait, were tested apart and first ran together in the checkpoint runs. LP decoding lowered P on both of its target instances on all four seeds: `item_placement` from 0.330 to 0.198 and `sing44` from 0.473 to 0.411 (test 12). The yielding wait let the LP relaxation on the EA's core finish about twice as early on `ca` (20.0 and 20.2 s instead of 37.3 and 38.3 s). It was adopted only after a follow-up, chosen once its first test had failed (test 11), lowered the bar on EA generations from 0.921 to 0.90 of the control's (test 15); its record estimates that this adds 12–19 percentage points to the chance of adopting a harmful change.

4.2 The checkpoint result

On four seeds, the coupled system had a lower P than cuOpt alone on 7 of the 11 instances of cuOpt's hard set and a higher one on 3 (Wilcoxon $p = 0.027$); the hint and the guard both passed their bars (Table 4). Over all 17 instances, P was lower on 8, higher on 8 and equal on 1. Figure 2 shows a typical instance.

The two halves agree: seeds 0–1 read 0.809 and +0.050, seeds 2–3 0.787 and +0.034. Seeds 2–3 ran only because seeds 0–1 passed the 0.95 bar, so they alone are the check against that selection.

Against SCIP alone, most of the drop on SCIP's hard set (0.889 to 0.448) is what cuOpt alone also reaches on the same cores and GPU (0.517); on these 15, the coupled system had a lower P

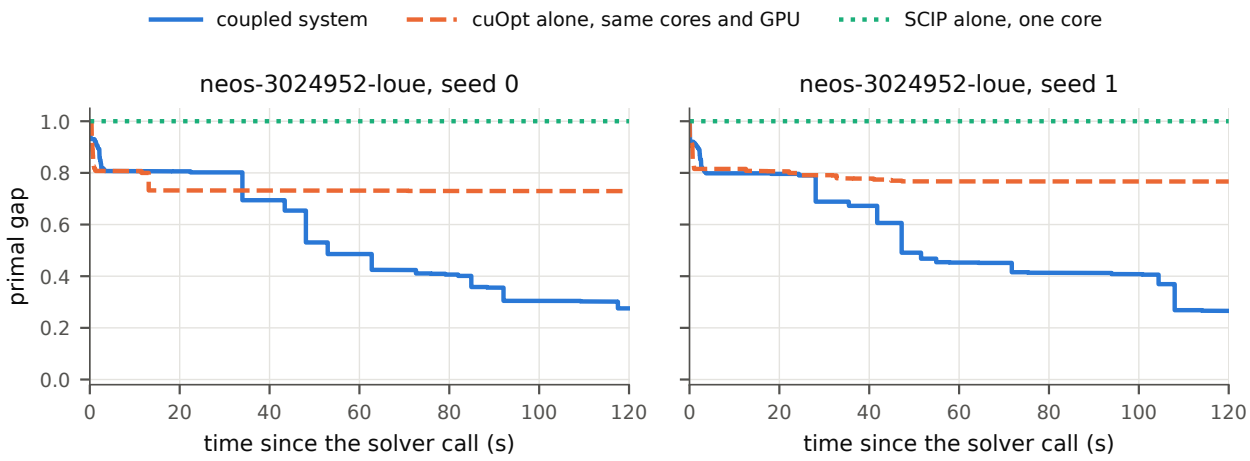


Figure 2: Gap of the best solution over time on `neos-3024952-loue`, seeds 0 and 1 of the checkpoint runs. P: coupled system 0.539 and 0.538, cuOpt alone 0.740 and 0.779, SCIP alone 1 (no solution in 120 s; 6 October). Chosen by a rule fixed before plotting (coupled-system P closest to the hard set's mean). Reference: MIPLIB's optimum, 26,756.

Table 4: The checkpoint result. Lower / higher: instances where the coupled system's mean P is lower / higher than the solver alone's; the rest are ties at $P = 1$ (no solution on either side). Mean P: solver alone → coupled system. Per instance: Appendix B.

runs	lower / higher	mean P	mean d [95 % CI]	hint	guard
against cuOpt alone, on cuOpt's hard set (11 instances)					
seeds 0–3 (15–16 Oct.)	7 / 3	0.690 → 0.550	+0.249 [+0.078, +0.441]	**0.798**	**+0.045**
seeds 0–3, crashes left out	6 / 3 of 10	0.689 → 0.547	+0.258 [+0.070, +0.463]	–	–
against SCIP alone, on SCIP's hard set (15 instances)					
seeds 0–3 (SCIP alone: 0–1)	12 / 0	0.889 → 0.448	+0.499 [+0.323, +0.676]		
rule				≤ 0.95	≥ -0.05

than cuOpt alone on 7, an equal one on 1 and a higher one on 7.

cuOpt alone's crashes favour the result: it ended with a segmentation fault on 3 of the 4 seeds of `comp21-2idx` (not on 13 October), and these runs count with P up to the crash (its one complete seed scored 0.520; seed mean 0.692, coupled 0.577), so that instance's gain rests on them. Without them that instance is incomplete, and the fixed rules read the hint and the guard only on complete sets; on the other 10 hard instances, P is lower on 6 and higher on 3 ($p = 0.055$).

The harm check (Table 2) flagged two instances. On `glass4` (its d_i −0.114, band 0.057), a later four-seed check in one window put three configurations level (mean P 0.218–0.223), so by a rule fixed beforehand the flag reads as day-to-day drift (test 18). Such drift can far exceed a band: `item_placement`'s P was 0.364 and 0.218 on two days with the same EA. On `comp21-2idx`, d_i was −0.051 against a band of 0.050, at the edge.

4.3 Where the system loses

The largest loss is `chromaticindex1024-7` (d −0.847 on four seeds), the largest negative term in the guard. Without the EA, cuOpt finds its first solution after 6.5–12.2 s; inside the coupled system, only after about 65 s (Table 5). There the EA's first generation is a 48-s GPU repair of about 1,300

back-to-back kernel launches, and neither SCIP's presence nor cuOpt's thread count explains the delay (test 13). Some ways of backing the EA off the GPU brought cuOpt back early, others did not (Table 5; tests 7, 9 and 14); why is not yet known.

Table 5: cuOpt's first solution on `chromaticindex1024-7` in every 120-s run that includes cuOpt, by what shares the GPU with it (7–16 October). Earlier harness: cuOpt's settings before the final harness's were adopted on 13 October (test 8). Per run: Appendix C.

what shares the GPU with cuOpt	runs	first (s)	P
nothing, or SCIP only (no EA)	22	6.5–12.2	0.054–0.102
the EA using the GPU freely (the coupled system, any configuration)	12	65.2–65.5	0.544–0.546
the EA on the GPU $\frac{1}{4}$ or $\frac{1}{2}$ of the time, or backing off while it adds nothing (earlier harness)	4	8.3–12.3	0.069–0.103
the EA backing off until cuOpt's first solution (final harness, with or without its two cuOpt settings)	4	63.2–63.5	0.527–0.529
the EA on the GPU $\frac{1}{8}$ of the time (earlier harness)	1	62.3	0.519

The other seven losses to cuOpt alone are smaller (Appendix B). On `doon` and `rail01`, cuOpt alone found a solution on 2 and 1 of 4 seeds (after 24–114 s), the coupled system on none. On `doon`, where cuOpt alone also found none in two runs of 15 October, a two-seed diagnostic ruled out cuOpt's three-core share and SCIP's presence (test 13); `rail01` has not been diagnosed. No arm found a solution on `cryptanalysiskb128n5obj16`.

5 Related work

Each part of I-TREA II has prior art. Our search ran on 5 October 2026, before the first experiment; works without a legally readable full text are listed in Appendix F, not cited.

Keeping infeasible solutions. Deb's feasibility rules made "feasible first" the common default in constrained evolutionary optimisation [8, 11, 30]. Several methods keep infeasible solutions on purpose: ASCHEA steers the feasible share [2], FI-2Pop keeps a separate infeasible population [23], and Farmani and Wright single out infeasible solutions better than the best feasible one [13]. SMES keeps the best infeasible solution [29], IDEA reserves a fixed share for infeasible solutions [42], and I-TREA added three ranks and an interactive decision maker [34].

Heuristics working with exact solvers. Two-way exchange of incumbents between a heuristic and a branch-and-bound solver running in parallel dates back two decades [14, 33, 37, 38]. MIP solvers recombine solutions from their own pool [3, 17], racing solvers that share incumbents gain modestly [7, 39, 40], and ReXi runs local-search workers beside SCIP workers over a shared pool [28].

GPU heuristics. GPU LP solvers [1, 26], bound propagation [43] and LP-free local search [25, 27] are the building blocks of current GPU MIP heuristics. cuOpt combines them [10] and runs a GPU population with recombination beside its own branch-and-bound [9, 32]; its source prompted two of our tests (5 and 6). CHAP pools feasible and infeasible candidates of GPU and CPU heuristics [44]; others fix and propagate from LP solutions [22] or search many subproblems at once [36, 41].

Position. The two closest works couple differently. cuOpt's population keeps infeasible members but ranks them behind the best feasible one, and works only with cuOpt's own branch-and-bound.

ReXi shares a pool with SCIP on the CPU and keeps no infeasible solutions. Our search found no GPU population with a capped share of infeasible solutions better than the incumbent, coupled to an external MIP solver by two-way exchange. This is a negative search result, not a priority claim.

6 Limitations

The results describe the development instances; they do not show how the method does elsewhere.

- **Development data.** The 17 instances were chosen and tuned on (Section 4.1: on eight new ones an earlier version lost to cuOpt alone); the result rests on four seeds (SCIP alone: two).
- **The gain over SCIP alone is not attributed.** The coupled system contains a full cuOpt run, and the checkpoint has no SCIP–cuOpt pair without the EA.
- **Selection.** cuOpt's hard set was fixed on earlier runs of cuOpt alone on three cores (`item_placement`, 0.488, and `sing44`, 0.471, now sit just below 0.5); SCIP's, on the compared runs.
- **Concentration.** `neos-960392` carries 35 % of the summed P reduction on cuOpt's hard set (the ratio without it: 0.858). `neos-2657525-crna` is bimodal (± 0.04 on the ratio at two seeds).
- **Checker tolerance.** Our exact checker accepted some `glass4` solutions better than MIPLIB's proven optimum (one of them in the 13 October run the harm check compares against) and rejected 15 cuOpt solutions in 5 of the 30 calibration runs. A single acceptance rule for all arms is pending; its effect on P and on the 1.31 ratio below is not yet known.
- **One machine, a small share.** On five cores and the GPU, cuOpt alone's primal integral is 1.31 times that of GAMS's published cuOpt 26.08 run on a whole GB10 (MIPfeas scoring [5, 6], 30 instances; Appendix C), so our cuOpt-alone baseline is weaker than cuOpt on the whole GB10.
- **Not measured:** HiGHS as a partner.

7 Conclusion

Phase 1 ends with the author's decision to continue: the final configuration lowered P on most instances that are hard for each solver alone, with one large loss (Section 4.3). The two harm flags, cuOpt alone's crashes and that loss are carried forward as stated caveats. Still open: how much of the gain needs the EA rather than cuOpt's own solutions, and whether any of it holds on unseen instances, which the next phase tests on cuOpt 26.10 under rules fixed before its runs.

References

[1] David Applegate, Mateo Díaz, Oliver Hinder, Haihao Lu, Miles Lubin, Brendan O'Donoghue, and Warren Schudy. Practical large-scale linear programming using primal-dual hybrid gradient. In *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, 2021. URL `https://arxiv.org/abs/2106.04756`.

[2] Sana Ben Hamida and Marc Schoenauer. An adaptive algorithm for constrained optimization problems. In *Parallel Problem Solving from Nature – PPSN VI*, volume 1917 of *Lecture Notes in Computer Science*, pages 529–538. Springer, 2000. doi: 10.1007/3-540-45356-3_52. URL `https://inria.hal.science/inria-00001273/document`. Open copy: HAL inria-00001273.

[3] Timo Berthold. Primal heuristics for mixed integer programs. Diploma thesis, Technische Universität Berlin, 2006. URL https://opus4.kobv.de/opus4-zib/files/1029/Berthold_Primal_Heuristics_For_Mixed_Integer_Programs.pdf. Carried out at Zuse Institute Berlin; ZIB OPUS record 1029.

[4] Timo Berthold. Measuring the impact of primal heuristics. *Operations Research Letters*, 41(6):611–614, 2013. doi: 10.1016/j.orl.2013.08.007. URL https://webdoc.sub.gwdg.de/ebook/serien/ah/ZIB/ZR-13-17.pdf. Open copy: ZIB-Report 13-17 (March 2013); record https://opus4.kobv.de/opus4-zib/frontdoor/index/index/docId/4766.

[5] Michael Bussieck. Open source MIPFEAS benchmark on the MIPLIB2017 instances. In H. D. Mittelmann, Benchmarks for Optimization Software, 2026. URL https://plato.asu.edu/ftp/mipfeas.html. Results page of 16 September 2026, prepared with assistance from NVIDIA.

[6] Michael Bussieck and Steve Dirkse. Expanding the focus: Introducing the MIPfeas benchmark. GAMS blog, 2026. URL https://www.gams.com/blog/2026/03/expanding-the-focus-introducing-the-mipfeas-benchmark/. Posted 17 March 2026, updated 16 September 2026.

[7] Rodolfo Carvajal, Shabbir Ahmed, George L. Nemhauser, Kevin Furman, Vikas Goel, and Yufen Shao. Using diversification, communication and parallelism to solve mixed-integer linear programs. *Operations Research Letters*, 42(2):186–189, 2014. doi: 10.1016/j.orl.2013.12.012. URL https://optimization-online.org/wp-content/uploads/2013/05/3900.pdf. Open copy: preprint on Optimization Online (30 May 2013).

[8] Carlos A. Coello Coello. Theoretical and numerical constraint-handling techniques used with evolutionary algorithms: a survey of the state of the art. *Computer Methods in Applied Mechanics and Engineering*, 191(11–12):1245–1287, 2002. doi: 10.1016/S0045-7825(01)00323-1. URL https://delta.cs.cinvestav.mx/~ccoello/journals/coello02-02.pdf.gz. Open copy: author's final manuscript (gzipped PDF) on the author's CINVESTAV page.

[9] Akif Çördük and Piotr Sielski. GPU-accelerated evolutionary framework for primal heuristics. Talk, Mixed Integer Programming Workshop 2025, University of Minnesota, 3–6 June 2025, 2025. URL https://www.mixedinteger.org/2025/. Talk abstract only (speaker: A. Çördük; joint work with P. Sielski); no paper or slides published.

[10] Akif Çördük, Piotr Sielski, Alice Boucher, and Kumar Aatish. GPU-accelerated primal heuristics for mixed integer programming, 2025. URL https://arxiv.org/abs/2510.20499. Poster at the NeurIPS 2025 Workshop on GPU-Accelerated and Scalable Optimization (ScaleOpt).

[11] Kalyanmoy Deb. An efficient constraint handling method for genetic algorithms. *Computer Methods in Applied Mechanics and Engineering*, 186(2–4):311–338, 2000. doi: 10.1016/S0045-7825(99)00389-8. URL http://repository.ias.ac.in/9407/1/310.pdf. Open copy: author preprint (December 1998), Indian Academy of Sciences repository.

[12] Thomas J. DiCiccio and Bradley Efron. Bootstrap confidence intervals. *Statistical Science*, 11(3):189–228, 1996. doi: 10.1214/ss/1032280214. URL https://projecteuclid.org/journals/statistical-science/volume-11/issue-3/Bootstrap-confidence-intervals/10.1214/ss/1032280214.pdf. With discussion and rejoinder. Free to read at the publisher.

[13] Raziyeh Farmani and Jonathan A. Wright. Self-adaptive fitness formulation for constrained optimization. *IEEE Transactions on Evolutionary Computation*, 7(5):445–455, 2003. doi: 10.1109/TEVC.2003.817236. URL https://ndownloader.figshare.com/files/17069744. Open copy: author deposit in Loughborough University's repository (figshare article 9447755).

[14] José E. Gallardo, Carlos Cotta, and Antonio J. Fernández. On the hybridization of memetic algorithms with branch-and-bound techniques. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 37(1):77–83, 2007. doi: 10.1109/TSMCB.2006.883266. URL https://carloscotta.com/papers/gallardo07hybridization.pdf. Open copy: pre-publication draft on co-author C. Cotta's site; pagination differs from the published version.

[15] Maxime Gasse, Simon Bowly, Quentin Cappart, Jonas Charfreitag, Laurent Charlin, Didier Chételat, Antonia Chmiela, Justin Dumouchelle, Ambros Gleixner, Aleksandr M. Kazachkov, Elias Khalil, Pawel Lichocki, Andrea Lodi, Miles Lubin, Chris J. Maddison, Christopher Morris, Dimitri J. Papageorgiou, Augustin Parjadis, Sebastian Pokutta, Antoine Prouvost, Lara Scavuzzo, Giulia Zarpellon, et al. The machine learning for combinatorial optimization competition (ML4CO): Results and insights. In *Proceedings of the NeurIPS 2021 Competitions and Demonstrations Track*, volume 176 of *Proceedings of Machine Learning Research*, pages 220–231, 2022. URL https://proceedings.mlr.press/v176/gasse22a.html.

[16] Ambros Gleixner, Gregor Hendel, Gerald Gamrath, Tobias Achterberg, Michael Bastubbe, Timo Berthold, Philipp Christophel, Kati Jarck, Thorsten Koch, Jeff Linderoth, Marco Lübbecke, Hans D. Mittelmann, Derya Ozyurt, Ted K. Ralphs, Domenico Salvagnin, and Yuji Shinano. MIPLIB 2017: data-driven compilation of the 6th mixed-integer programming library. *Mathematical Programming Computation*, 13(3):443–490, 2021. doi: 10.1007/s12532-020-00194-3. URL https://www.or.rwth-aachen.de/files/research/publications/Gleixner2021_Article_MIPLIB2017.pdf. Open access, CC BY 4.0.

[17] Gregor Hendel. Adaptive large neighborhood search for mixed integer programming. *Mathematical Programming Computation*, 14(2):185–221, 2022. doi: 10.1007/s12532-021-00209-7. URL https://link.springer.com/content/pdf/10.1007/s12532-021-00209-7.pdf. Open access, CC BY 4.0. Earlier version: ZIB-Report 18-60, https://opus4.kobv.de/opus4-zib/frontdoor/index/index/docId/7116.

[18] HiGHS development team. HiGHS: High-performance parallel linear optimization software (LP, MIP, QP). https://highs.dev/, 2026. MIT licence; release 1.15.1 used here. Source: https://github.com/ERGO-Code/HiGHS.

[19] Christopher Hojny, Mathieu Besançon, Ksenia Bestuzheva, Sander Borst, João Dionísio, et al. The SCIP optimization suite 10.0, 2025. Preprint: https://arxiv.org/abs/2511.18580.

[20] Q. Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018. doi: 10.1007/s12532-017-0130-5. Open access (CC BY 4.0): https://link.springer.com/article/10.1007/s12532-017-0130-5.

[21] IEEE PES Task Force on Benchmarks for Validation of Emerging Power System Algorithms. PGLib-UC: Benchmarks for the unit commitment problem. GitHub repository power-grid-lib/pglib-uc, release v19.08 (7 August 2019), 2019. URL https://github.com/power-grid-lib/pglib-uc. Data licence CC BY 4.0. Cite also the original sources of the cases used, as the repository asks.

[22] Nils-Christian Kempke and Thorsten Koch. Fix-and-propagate heuristics using low-precision first-order LP solutions for large-scale mixed-integer linear optimization. *Mathematical Programming Computation*, 2026. doi: 10.1007/s12532-026-00312-7. URL https://link.springer.com/content/pdf/10.1007/s12532-026-00312-7.pdf. Open access, CC BY 4.0; published online 6 July 2026. Preprint also ZIB-Report 25-04.

[23] Steven Orla Kimbrough, Gary J. Koehler, Ming Lu, and David Harlan Wood. On a feasible–infeasible two-population (FI-2Pop) genetic algorithm for constrained optimization: Distance tracing and no free lunch. *European Journal of Operational Research*, 190(2):310–327, 2008. doi: 10.1016/j.ejor.2007.06.028. URL https://repository.upenn.edu/server/api/core/bitstreams/a83719e3-9b32-4200-876f-8eb9da988225/content. Open copy: author manuscript of 10 April 2007, University of Pennsylvania repository (CC BY-NC-ND 4.0); wording and pagination may differ.

[24] Bernard Knueven, James Ostrowski, and Jean-Paul Watson. On mixed-integer programming formulations for the unit commitment problem. *INFORMS Journal on Computing*, 32:857–876, 2020. doi: 10.1287/ijoc.2019.0944. URL https://optimization-online.org/wp-content/uploads/2018/11/6930.pdf. Volume and pages from DBLP/Semantic Scholar (not in Crossref); issue not verified. Open copy: preprint, Optimization Online (November 2019 revision).

[25] Peng Lin, Mengchuan Zou, and Shaowei Cai. An efficient local search solver for mixed integer programming. In *30th International Conference on Principles and Practice of Constraint Programming (CP 2024)*, volume 307 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 19:1–19:19. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024. doi: 10.4230/LIPIcs.CP.2024.19. URL https://drops.dagstuhl.de/storage/00lipics/lipics-vol307-cp2024/LIPIcs.CP.2024.19/LIPIcs.CP.2024.19.pdf. Open access, CC BY 4.0.

[26] Haihao Lu and Jinwen Yang. cuPDLP.jl: A GPU implementation of restarted primal-dual hybrid gradient for linear programming in Julia. *Operations Research*, 73(6):3440–3452, 2025. doi: 10.1287/opre.2024.1069. URL https://arxiv.org/abs/2311.12180.

[27] Bjørnar Luteberget and Giorgio Sartor. Feasibility jump: an LP-free Lagrangian MIP heuristic. *Mathematical Programming Computation*, 15(2):365–388, 2023. doi: 10.1007/s12532-023-00234-8. URL https://link.springer.com/content/pdf/10.1007/s12532-023-00234-8.pdf. Open access, CC BY 4.0. Author preprint: https://luteberget.github.io/preprints/feasibilityjump-2023-01-09.pdf.

[28] Gioni Mexi and Daniel Rehfeldt. Race, exchange, improve: Finding high-quality MIP solutions quickly, 2026. URL https://arxiv.org/abs/2609.05954.

[29] Efrén Mezura-Montes and Carlos A. Coello Coello. A simple multimembered evolution strategy to solve constrained optimization problems. *IEEE Transactions on Evolutionary Computation*, 9(1):1–17, 2005. doi: 10.1109/TEVC.2004.836819. URL https://delta.cs.cinvestav.mx/~ccoello/journals/tec839-final.pdf.gz. Open copy: authors' final accepted manuscript (gzipped PDF) on the author's CINVESTAV page.

[30] Efrén Mezura-Montes and Carlos A. Coello Coello. Constraint-handling in nature-inspired numerical optimization: Past, present and future. *Swarm and Evolutionary Computation*, 1(4):173–194, 2011. doi: 10.1016/j.swevo.2011.10.001. URL https://delta.cs.cinvestav.mx/~ccoello/journals/swevo-mezura.pdf.gz. Open copy: authors' preprint (gzipped PDF) on the author's CINVESTAV page.

[31] NVIDIA. NVIDIA cuOpt: GPU-accelerated decision optimization (LP, MILP, routing). https://github.com/NVIDIA/cuopt, 2026. Open source (Apache-2.0); release 26.08 used here.

[32] NVIDIA cuOpt developers. NVIDIA cuOpt: GPU-accelerated optimization engine (MIP heuristics: population, recombiners and diversity manager). Source code, GitHub repository NVIDIA/cuopt, Apache License 2.0, 2026. URL https://github.com/NVIDIA/cuopt/tree/c2f3df54af31d51c106791812ad1829ab0ca5caa/cpp/src/mip_heuristics/diversity. Read at commit c2f3df54af31d51c106791812ad1829ab0ca5caa (5 October 2026), and at commit dfbbcc5a7693968144531f5d6a3b7fe38102ebd5 (8 October 2026) for the population's add-solution and clearing rule; user guide version 26.08.

[33] V. Nwana, K. Darby-Dowman, and G. Mitra. A co-operative parallel heuristic for mixed zero-one linear programming: Combining simulated annealing with branch and bound. *European Journal of Operational Research*, 164(1):12–23, 2005. doi: 10.1016/j.ejor.2002.12.002. URL https://bura.brunel.ac.uk/handle/2438/702. Open copy: preprint of 15 June 2003, Brunel University Research Archive.

[34] Berk Orbay. An interactive evolutionary algorithm for the multiobjective relocation problem with partial coverage. M.s. thesis, operational research, Middle East Technical University, Ankara, Turkey, April 2011. URL https://open.metu.edu.tr/handle/11511/20509. Supervisor: Esra Karasakal. Handle hdl:11511/20509; open PDF https://open.metu.edu.tr/bitstream/handle/11511/20509/index.pdf.

[35] Berk Orbay. Optimizing the optimizers: Measuring learned control of MIP solvers on a single DGX spark, 2026. URL https://doi.org/10.5281/zenodo.22947442. Technical report; code at https://github.com/berkorbay/optopt.

[36] Jean Pauphilet and Yupeng Wu. PaNGEA: Parallel node generation and exploration algorithm on GPU, 2026. URL https://arxiv.org/abs/2610.03090.

[37] Jakob Puchinger and Günther R. Raidl. Combining metaheuristics and exact algorithms in combinatorial optimization: A survey and classification. In J. Mira and J. R. Álvarez, editors, *Artificial Intelligence and Knowledge Engineering Applications: A Bioinspired Approach (IWINAC 2005, Part II)*, Lecture Notes in Computer Science, pages 41–53. Springer, 2005. doi: 10.1007/11499305_5. URL https://www.ac.tuwien.ac.at/files/pub/puchinger-05.pdf. Open copy: author version, TU Wien. LNCS volume number not verified.

[38] Jakob Puchinger, Günther R. Raidl, and Martin Gruber. Cooperating memetic and branch-and-cut algorithms for solving the multidimensional knapsack problem. In *MIC 2005: The 6th Metaheuristics International Conference*, pages 775–780, Vienna, Austria, 2005. URL https://www.ac.tuwien.ac.at/files/pub/puchinger-05a.pdf.

[39] Ted Ralphs, Yuji Shinano, Timo Berthold, and Thorsten Koch. Parallel solvers for mixed integer linear optimization. In *Handbook of Parallel Constraint Reasoning*, pages 283–336. Springer, 2018. doi: 10.1007/978-3-319-63516-3_8. URL https://optimization-online.org/wp-content/uploads/2017/05/5993.pdf. Open copy: COR@L Technical Report 16T-014-R3 / ZIB-Report 16-74, Optimization Online.

[40] Yuji Shinano, Stefan Heinz, Stefan Vigerske, and Michael Winkler. FiberSCIP – a shared memory parallelization of SCIP. *INFORMS Journal on Computing*, 30(1):11–30, 2018. doi: 10.1287/ijoc.2017.0762. URL `https://optimization-online.org/wp-content/uploads/2017/04/5944.pdf`. Open copy: ZIB-Report 13-55 (third revision), Optimization Online.

[41] Kyuil Sim, Sanghyeok Choi, and Jinkyoo Park. Solving integer linear programming with parallel tempering, 2026. URL `https://arxiv.org/abs/2605.29366`.

[42] Hemant Kumar Singh. *Development of optimization methods to deal with current challenges in engineering design optimization*. PhD thesis, School of Engineering and Information Technology, University College, University of New South Wales at the Australian Defence Force Academy, 2011. URL `https://unsworks.unsw.edu.au/bitstreams/251ed4c1-33f6-4162-a54c-2e45d700f0ff/download`. Open access (UNSWorks, CC BY-NC-ND 3.0 AU). Chapter 3, Algorithm 3.1 gives IDEA in full.

[43] Boro Sofranac, Ambros Gleixner, and Sebastian Pokutta. Accelerating domain propagation: an efficient GPU-parallel algorithm over sparse matrices. *Parallel Computing*, 109:102874, 2022. doi: 10.1016/j.parco.2021.102874. URL `https://arxiv.org/abs/2009.07785`. Article 102874.

[44] Gennesaret Kharistio Tjusila, Alexander Hoen, Nils-Christian Kempke, Gioni Mexi, Timo Berthold, Ambros Gleixner, Thorsten Koch, and Sebastian Pokutta. CHAP: A hybrid GPU-CPU heuristic for MIP, 2026. URL `https://arxiv.org/abs/2605.05086`.

A Development record

Table 6 lists every test of phase 1, one row per decision record, numbered as in the main text. Test 8 is the checkpoint's first reading (earlier configuration, 13 October); tests 16 and 17 are the two halves of the result, and test 17 reads all four seeds. The framework records (D-001 to D-010, D-013, D-019 to D-021, D-024) and the go at C1 (D-033) are, like all records with their amendments and dated corrections, in the repository (`vault/50-decisions/`).

Exceptions to rules fixed before the runs (each stated in its record): tests 1, 2 and 15, in Table 3's notes a–c and Section 4.1 (test 2's two deviations: a criterion built on a premise a one-variable test refuted, and a mis-set band; test 15's thresholds were fixed with seeds 0–1 known); test 6, adopted although one check that it worked as intended failed by its letter (the decoder's host time on one instance, a cost that could only handicap the treatment); and test 8, whose amendment fixing the checkpoint's hint and guard came after 7 of 58 runs had finished, none read.

B Per-instance checkpoint results

Table 7 gives the phase-1 result per instance, under rules fixed before the runs (D-022, D-025, D-030, D-031). VALID: 136 of 136 runs, cuOpt alone's three crashed runs kept as outcomes.

Caveats: Sections 4.2 and 6, and in the repository D-033.

C Measurement harness and calibration

Every compared GPU arm gets the same machine share F (Table 8). Thermal rules are part of the measurement, since a capped GPU would bias a comparison; the runner enforces them (D-003, D-008, D-009, D-021). The 120-s budget starts at a shared barrier, after every process has read the instance and warmed its GPU kernels.

Thermal rules. The host's fan governor caps the GPU clock at 2,200 MHz after 20 s at or above 85 °C at maximum fan, and requests a 30-minute rest after 30 minutes at maximum fan.

- **Windows and start gate:** daytime windows under a lease, with the local language-model servers stopped. A GPU run starts below 80 °C (CPU-only: 88 °C), with no rest requested and no clock cap; rests took about a quarter of window time.
- **Voids:** a block (one instance and seed, both arms) is voided, requeued once and rerun whole if the hottest zone reaches 92 °C, a GPU run's median busy SM clock is below 2,300 MHz or meets a clock cap, another program's GPU process runs for over 2 % of the budget, or a kernel compiles on the clock. Below 2,000 MHz (a known stuck-clock fault) the run set halts.
- **Breaker:** two thermal or cap voids close the window. In the C1 re-read it fired once (15 October, D-030); the remaining blocks ran whole the next morning.

Harness measurements (D-021), made before any run on unseen instances; they test the harness, not the method. M4 reads outside its band, which D-021 attributes to F (five threads and the GPU, against a 24-thread whole machine) and to the machine, so the unseen-instance test's targets are anchored on our own cuOpt-alone runs.

- **M1 smoke** (12 runs of 120 s): every harness setting acts as specified; unsetting cuOpt's thread variables is inert (cuOpt CPU-s per wall-s 5.02 / 4.99 / 5.03 unset, 5.02 / 4.98 / 5.03 set). Passed.
- **M2 soak** (4 runs of 600 s on F): steady-state p95 66–73 °C, maximum 76 °C, no sample $\geq$ 90 °C, no clock cap, busy SM clock 2,392 MHz. Passed; F accepted.
- **M3 memory** (8 instances, cuOpt alone at 40-GiB caps, 600 s): `eilA101-2` and `nw04` (cap hits) and `supportcase6` (peak $> 0.8 \times 24$ GiB) do not fit F; the unseen test set goes from 218 to 215 instances.

no.	record	question	inst. × seeds (runs)	**verdict**: key numbers
1	D-011 (M)	SCIP streams good infeasible node-LP points to the EA (A3)	9 × 2 (36)	**kept on, not credited**: d +0.05; 3 / 1
2	D-012 (T)	Repair depth set per generation: the repair clock (A3)	9 × 2–4 (44)	**default; 2 rules failed by the letter**: `item_placement_9903` +0.50 (4 of 4 seeds); all 5 / 2, +0.12 [+0.00, +0.26]
3	D-014 (T)	Does A3 hold on 8 new instances (predictions fixed first)?	8 × 2 (48)	**INCONCLUSIVE (3 of 5)**: vs cuOpt alone 1 / 5, −0.51 [−0.84, −0.12]
4	D-015 (pilot)	Pass cuOpt's solutions to the EA: arm A9	7 × 1 (21)	**built**: vs SCIP alone 7 / 0, +0.68 [+0.42, +0.90]; vs cuOpt alone 3 / 4, −0.07 [−0.42, +0.25]
5	D-016 (T)	Rank 1 by cuOpt's clearing rule (A3)	9 × 2, 8 × 2 (102)	**NOT ADOPTED**: d −0.033 [−0.122, +0.050]; 4 / 4
6	D-017 (T)	LP decode of candidates' continuous variables (A3)	6 × 2 (24)	**ADOPTED (A3); 1 rule failed by the letter**: d +0.219 [+0.041, +0.429]; 5 / 1; `item_placement_9903` 0.311 → 0.118
7	D-018 (T)	The EA backs off the GPU while it does not contribute (A9)	14 × 2 (84)	**NOT ADOPTED: 3 mechanism rules failed**: outcome rules held; `chromaticindex1024-7` d +0.834; all 9 / 5, +0.083 [−0.054, +0.232]
8	D-022 (T, R)	A9 under the final harness; C1's first reading	17 × 2 (74)	**ADOPT (mechanism vacuous)**: no harm where cuOpt can stop itself (3); old default: hint 0.827, guard +0.063
9	D-023 (T)	Phase Y only: the EA yields until cuOpt's first incumbent	11 × 2 (66)	**NOT ADOPTED: 4 rules failed**: `chromaticindex1024-7` d +0.030 (needed ≥ +0.30); cuOpt first at 63.47 / 63.41 s
10	D-025 (T)	R1: the EA's CUDA context blocks instead of spinning	5 × 2 (20)	**NOT ADOPTED**: hint 0.765; generations 0.824, 0.818 (bars 0.921, 0.950); `comp21-2idx` ΔP +0.060 (band 0.050)
11	D-025 (T)	R1 fallback: the CUDA context yields instead of spinning	5 × 2 (20)	**NOT ADOPTED**: hint 0.834; `item_placement_9903` generations 0.759 (bar 0.921): 678 vs 1,480 on seed 1
12	D-026 (T)	R2: LP decode inside A9, after the EA's LP worker	2 × 4, 3 × 2 (28)	**ADOPT**: hint 0.758 (bar 0.90); `item_placement_9903` 0.330 → 0.198, `sing44` 0.473 → 0.411, all 4 seeds
13	D-027 (D)	Why A9 loses on 4 instances: cuOpt's threads or SCIP?	4 × 2 (22)	**the EA's presence**: cuOpt's first incumbent, `chromaticindex1024-7`: 6.5–9.8 s without the EA, 62–66 s in A9
14	D-028 (D)	Do R13 and R14 keep phase Y from helping cuOpt?	1 × 2 (4)	**not R13 and R14**: 63.25 / 63.24 s against a limit of 19.82 s
15	D-029 (T)	The yield flag again, on seeds 0–5 with D-025's failing seed	3 × 2–4 (20; 28 read)	**ADOPT; a bar loosened after the failure**: generations 1.020, 1.152 (bar 0.921 → 0.90); ΔP +0.016, +0.015 (bar +0.05); `ca__Scenario400_reserves_1` P ratio, seeds 2–3: 0.675 (bar 0.95)
16	D-030 (R)	C1 on the frozen-v2 candidate, cuOpt alone in the same window	17 × 2 (68)	**VALID; harm flag: `glass4`**: hint 0.809, guard +0.050; cuOpt alone crashed on `comp21-2idx` (kept)
17	D-031 (R)	The phase-1 result: C1 at seeds 2–3, read with D-030's 0–1	17 × 2 (68; 136 read)	**VALID; harm flags: `glass4`, `comp21-2idx`**: hint 0.798, guard +0.045 (seeds 2–3: 0.787, +0.034); vs cuOpt alone 7 / 3, +0.249 [+0.078, +0.441]
18	D-032 (T)	`glass4`'s harm flag: real, or day-to-day drift?	1 × 4 (12)	**DRIFT: candidate freezes**: mean P 0.2176 / 0.2233 / 0.2177 (3 configurations); d −0.025 (bar −0.057)

Table 6: The development record of phase 1. no.: the test number of the main text. Kinds: T, a test with rules fixed before any of its runs was read (exceptions in the text); M, measured without such a rule; D, a diagnostic; R, a C1 reading. a / b: instances better / worse; d: mean bounded difference (positive: the change or A9 better) [95% bootstrap interval]; hint: ratio of mean $P(120)$; generations: treatment over control. 120-s runs, 1–6 seeds: development data, descriptive.

- **M4 calibration** (D-020's 30-instance sample, cuOpt alone on F, 600 s), against GAMS's published cuOpt 26.08 run on a GB10: late-clock check passed (median first incumbent, ours − GAMS, −0.119 s, n 10); R = 1.313 [1.091, 1.618] on the call clock (launch clock 1.963 [1.606, 2.408]), OUTSIDE the band [0.870, 1.150]; feasible 27 vs GAMS 27. R: ratio of shifted geometric means (shift 0.001) of GAMS's MIPfeas primal integral, ours over GAMS's, with a 95% bootstrap interval. Our checker rejected 15 of cuOpt's incumbents in 5 of the 30 runs; GAMS's scoring would count them, and their effect on R has not been computed.
- **M6 lanes** (20 runs): SCIP alone's nodes/s are 13.2% lower beside 3 busy lanes (rule: at most 3%); hottest zone 78.6 °C. Verdict: 3 lanes.

Table 9 gives the per-run values behind Table 5.

D Reproducibility

Every number here comes from the code, configurations and job files in the project repository (to be made public with the final version). The chain, with analysis steps pinned (`OMP_NUM_THREADS=1 nice -n 19 taskset -c 10-14`);

		$P(120)$, seed means			d by half			harm check
instance	set	A0	A6F	A9	d	seeds 0–1	seeds 2–3	d_i / band
cuOpt's hard set, the fixed 11 (the hint reads these)								
`neos-960392`	dev	1.000	0.584	0.042	+0.929	+0.935	+0.921	+0.281 / 0.411
`item_placement_9903`	dev	0.660	0.488	0.220	+0.550	+0.469	+0.625	+0.396 / 0.050
`ca__Scenario400_reserves_1`	dev	1.000	0.555	0.261	+0.529	+0.550	+0.508	+0.337 / 0.050
`neos-2657525-crna`	probe	0.833	0.647	0.428	+0.339	+0.180	+0.449	–
`neos-3024952-loue`	dev	1.000	0.755	0.523	+0.308	+0.291	+0.325	+0.072 / 0.050
`sing44`	dev	0.525	0.471	0.387	+0.178	+0.295	+0.062	+0.131 / 0.128
`comp21-2idx`	probe	0.891	0.692[a]	0.577	+0.166	+0.272	+0.029	−0.051 / 0.050*
`cryptanalysiskb128n5obj16`	probe	1.000	1.000	1.000	0	0	0	–
`rail01`	dev	1.000	0.979	1.000	−0.021	−0.042	0	–
`neos-4763324-toguru`	probe	0.772	0.553	0.612	−0.097	−0.090	−0.104	−0.015 / 0.086
`neos-2746589-doon`	probe	1.000	0.861	1.000	−0.139	−0.278	0	–
the other 6 (the guard reads all 17)								
`glass4`	dev	0.344	0.319	0.244	+0.237	+0.232	+0.241	−0.114 / 0.057*
`neos-3656078-kumeu`	probe	1.000	0.085	0.107	−0.204	−0.272	−0.121	−0.040 / 0.294
`neos-3004026-krka`[†]	probe	0.659	0.003	0.004	−0.279	−0.077	−0.540	−0.156 / 0.240
`radiationm18-12-05`[†]	dev	0.229	0.008	0.011	−0.289	−0.231	−0.336	−0.042 / 0.050
`fhnw-binpack4-48`[†]	dev	1.000	0.005	0.011	−0.591	−0.528	−0.642	+0.004 / 0.458
`chromaticindex1024-7`	probe	1.000	0.083	0.544	−0.847	−0.860	−0.834	+0.001 / 0.185
hint: mean P, fixed 11; ratio			0.690	0.550	0.798	0.809	0.787	
guard: mean d over the 17					+0.045	+0.050	+0.034	

Table 7: The 17 instances and the phase-1 result per instance: seed means of $P(120)$. A0: SCIP alone, seeds 0–1 (6–7 October). A6F: cuOpt alone; A9: the coupled system; same cores, GPU and window, seeds 0–3 (15–16 October). d: A9 against A6F, positive if A9 is lower. Harm check: against A9 of 13 October; * flagged, – cannot fire. Set: development set or class probe; all MIPLIB 2017 except `ca__Scenario400_reserves_1` (PGLib-UC) and `item_placement_9903` (ML4CO). [a]Crashed on seeds 0–2 (P 0.776, 0.782, 0.690, kept); 0.520 on seed 3. [†]Both arms at $P \leq 0.011$.

Table 8: The machine share F under the final harness. Memory caps are hard (systemd `MemoryMax` on the host, a pool limit on the GPU); the runner's own threads run outside F.

process	cores	memory caps and settings
SCIP 10, one thread	Cortex-X925 core 5	host 9 GiB (SCIP's own limit 8 GiB)
EA and its two HiGHS processes	Cortex-X925 core 6; GPU	host 6 GiB; GPU pool 12 GiB; CUDA yield flag
cuOpt 26.08, three threads	Cortex-A725 cores 1–3; GPU	host and GPU 24 GiB each; MIP gap limits 0; thread variables (OMP, OpenBLAS, MKL) unset; own kernel cache
cuOpt alone, five threads	all five cores; GPU	as cuOpt above

what shares the GPU with cuOpt	runs	cuOpt's first solution (s), by decision record
nothing, or SCIP only (no EA)	22	D-014: 8.56, 9.44; D-015: 6.50; D-018: 8.12, 10.15, 9.69; D-022: 6.53, 8.01; D-023: 10.87, 10.52; D-027: 6.53, 9.65, 9.24, 8.21, 9.80, 7.33; D-028: 7.45, 9.91; D-030: 7.94, 10.31; D-031: 9.36, 12.23
the EA using the GPU freely (the coupled system, any configuration)	12	D-015: 65.35; D-018: 65.30, 65.29, 65.46; D-022: 65.26, 65.30; D-023: 65.31, 65.46; D-030: 65.24, 65.22; D-031: 65.24, 65.23
the EA on the GPU $\frac{1}{4}$ or $\frac{1}{2}$ of the time, or backing off while it adds nothing (earlier harness)	4	D-018: 9.81, 8.33, 9.38, 12.35
the EA backing off until cuOpt's first solution (final harness, with or without its two cuOpt settings)	4	D-023: 63.47, 63.41; D-028: 63.25, 63.24
the EA on the GPU $\frac{1}{8}$ of the time (earlier harness)	1	D-018: 62.29

Table 9: cuOpt's first checked solution (seconds since the solver call) on `chromaticindex1024-7` in every 120-s run with cuOpt, 7–16 October, grouped as in Table 5.

Table 10: Parameters of the final configuration: (f) set in the file, others code defaults (`src/itrea/config.py`).

part	value	part	value
population	256 (fewer if it exceeds the GPU pool); 256 children per generation	ranks	Rank 0 at most 10 % of the population; Rank 1 the best 10 % of the feasible
selection	binary tournament: Rank 1, 0, 2, then objective, then violation	variation	crossover 0.9 (agreement 0.6, uniform 0.2, constraint block 0.2); mutation of Poisson(4) integers
repair	feasibility jump, at most 20,000 moves, stop after 1,000 without gain (f)	repair clock	active (f); horizons 2^7–2^{16} steps, or none
strategies	four (row scale × LP guidance) (f); redraw 0.1; each at least 5 %	stagnation	after 50 generations, 25 % of Rank 2 replaced by immigrants
EA's LP	HiGHS on the EA's core (f), 60-s limit	LP stream	on (f); at most 2 % of SCIP's time; 16 points, 2 per tick, 4 children each
LP decoding	on (f), after the EA's LP and at its priority (f); up to 4 LPs per batch, 2 s (infeasible 1 s, first 10 s)	exchange	mailbox polled every 0.25 s; up to 8 SCIP pool solutions within 1 %; cuOpt's incumbents
GPU wait	CUDA yield flag (f)	other	frontier of 32 (output only); feasibility tolerance 10^{-6}; GPU pool 12 GiB

each test's job file and reading command are in its decision record and the experiment index:

1. `itrea make-jobs` (or a test's tool) writes `jobs/*.jsonl`: instance, arm, seed, budget, configuration, harness, set, block.
2. `itrea pool jobs/<file>.jsonl` writes `traces/raw/<set>/<run-id>/`: run record, logs, every incumbent.
3. `itrea build-runs` checks every incumbent and writes `datasets/runs.parquet` and `datasets/incumbents_checked.parquet`.
4. The C1 readings are saved by `itrea c1r-report --a9 ysync-ld0 --json logs/d030-reading.json`, then `tools/d031_report.py --d030-json logs/d030-reading.json --json logs/d031-reading.json`.
5. `tools/paper_tables.py` writes `paper/generated/` and `paper/figures/`; `--check` compares byte for byte.

Runs and the checker. A run record holds the configuration hashes, harness, cores, caps, versions, revision, a clean-tree flag and temperatures. The checker re-reads the model with another library, decides bounds, integrality and objective exactly, and re-checks rows exactly where floating point could be wrong (tolerance 10^{-6}, all arms); only passing incumbents enter P. Run table of 16 October, 13:01: 1,479 runs (1,469 completed, 10 failed and scored up to the failure); 28 of 192,602 incumbents, in 13 runs, failed. In the four C1 run sets (222 runs, all from a clean tree), 18,098 of 18,100 passed.

The final configuration (frozen at the checkpoint). `configs/frozen-v2.yaml`, a byte copy of `configs/itrea2-dev-rc-3way-ysync-ld0.yaml` (SHA-256 `d5907fc3...`): EA hash `835c7c5a249c`, A9 run hash `f345de0ab936` (the last field of every A9 run id). Run hashes of cuOpt alone and SCIP alone: `82a371305f88`, `27f3171fac78`. A repository test fails if the frozen file carries another hash pair. Table 10 lists its parameters.

Re-analysis and replication. A re-analysis must match exactly (`itrea reproduce` covers the pre-registered chain, not yet the per-test reports or this table writer); a replication will differ (wall-clock metrics, GPU arithmetic, thermal rests). Provisional references: the study's best on the unit-commitment instance (it re-scores those runs) and the screening's best on the item-placement instance.

Environment. Ubuntu 24.04, kernel 6.17.0-1031-nvidia, driver 580.173.02, CUDA 13.0; Python 3.12.3, NumPy 2.4.6, SciPy 1.18.1, CuPy 14.2.0, HiGHS 1.15.1, PySCIPOpt 6.2.1 (SCIP 10.0), cuOpt 26.08, pandas 3.0.3, PyArrow 23.0.1 (pinned in `pyproject.toml`; each run record stores its versions).

Not yet public. The repository is private until the final version. Raw runs (`traces/raw/`, about 3.5 GB) and run tables are to be published as a versioned archive, as for the predecessor study [35]; step 4 regenerates the readings in `logs/`. Instances and GAMS's archive are third-party data. Open (issue #36): the checker accepted incumbents below MIPLIB's optimum on `glass4` (11 runs, two in the C1 sets) and `mas74`; one rule for all arms is due before the second pre-registration.

E AI assistance statement

Who did what. The author set the question and the direction and took the decisions that changed the study, among them the go at checkpoint C1 (D-033). AgentRA (Agent Research Assistant: Claude Code with Anthropic's Claude Opus 5.5) did the rest under his direction: design and pre-registration, the `itrea` package and tests, job files, every timed run, report tools and analyses, decision records, the journal and this report's text. Each record names who decided it and quotes the author's directing statements. The author is responsible for the content. Large

tasks ran as Claude Code workflows (a main session and subagents of the same model on disjoint files: builders, adversarial reviewers, refuters and fixers; editors and fact-checkers). Twice, with the author's permission, AgentRA asked OpenAI's GPT-6.1 Sol, through the Hermes Agent harness, a design question, sharing only the design, public instance names and our measurements.

Limits. The reviewers are instances of the agent's own model and can share its blind spots. No person or model of another family has reviewed this report or its results. The final version will add the counts of this report's own checks and any external review. Errors were caught by the author (three operational failures), adversarial checks (test-rule defects before any run was read), run records (a block with kernels compiled on the clock was voided) and fact-checks (D-030's crashed runs first reported at $P = 1$ score 0.776 and 0.782). A subagent once sent the author's e-mail address to an outside service unasked; prompts now forbid it. Corrections and incidents are recorded, dated, in the repository.

F Works without a legal free copy

Table 11 lists relevant works behind paywalls, with no legal free copy found. *We have not read them in full*; what we say rests on abstracts, metadata and the open works cited instead.

Table 11: Works we could not read in full, with the open work cited instead (–: none in this report).

work	DOI	cited instead
Runarsson and Yao (2000), stochastic ranking, IEEE TEVC	10.1109/4235.873238	[30][a]
Ray et al. (2009), IDEA, book chapter	10.1007/978-3-642-00619-7_7	[42]
Singh et al. (2008), IDEA, AI 2008	10.1007/978-3-540-89378-3_11	[42]
Isaacs et al. (2008), CHEA, IDEA's precursor, IEEE CEC	10.1109/CEC.2008.4631171	[42]
Kimbrough et al. (2003), two-population GA, GECCO	10.1007/3-540-45105-6_123	[23]
Takahama and Sakai (2006), ε-constrained differential evolution, CEC	10.1109/CEC.2006.1688283	[30]
Rothberg (2007), evolutionary polishing of MIP solutions, IJOC	10.1287/ijoc.1060.0189	[3, 28]
Danna et al. (2005), relaxation-induced neighbourhoods, Math. Prog.	10.1007/s10107-004-0518-7	[3, 17]
French et al. (2001), hybrid GA and branch-and-bound, J. Heuristics	10.1023/A:1011921025322	[37]
Fischetti and Fischetti (2018), Matheuristics, book chapter	10.1007/978-3-319-07124-4_14	–
Boschetti et al. (2009), Matheuristics, HM 2009	10.1007/978-3-642-04918-7_13	–
Davies et al. (2024), ViolationLS: feasibility jump in CP-SAT	10.1007/978-3-031-60597-0_16	–
Lodi and Tramontani (2013), MIP performance variability, TutORials	10.1287/educ.2013.0112	–[b]

[a] A comparator option in our code; checked in an archived former author page, whose legal status is open. [b] Reported free at the publisher; not verified.